Parallel Evolutionary Portfolio Optimizer

Island Model Genetic Algorithm with OpenMP

A Project Component for

Parallel and Distributing Computing (UCS645)

By

Sr	Name	Roll No
1	Subhendu Sharma	102303013

Under the guidance of

Dr. Saif Nalband

(Assistant Professor, DCSE)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

(DEEMED TO BE UNIVERSITY)

PATIALA - 147004

APRIL, 2026

Table of Contents

1 Introduction **1**

1.1 Background 1

1.2 Problem Statement 1

1.3 Objectives 2

2 Literature Review **3**

3 Research Gaps **5**

4 Methodology **6**

4.1 Problem Formulation 6

4.2 Island Model Genetic Algorithm (IMGA) 6

4.3 Migration Topologies 8

4.3.1 Ring Topology 8

4.3.2 Star Topology 8

4.4 OpenMP Parallelization Strategy 8

4.4.1 Task Parallelism: Inter-Island 8

4.4.2 Data Parallelism: Intra-Island Fitness Evaluation 9

4.5 Serial Phases (Synchronization Points) 9

5 Implementation Details **10**

5.1 Technology Stack 10

5.2 Key Data Structures 10

5.3 Reproducibility Measures 10

5.4 CLI Parameters 11

6 Datasets **12**

6.1 Data Source 12

6.2 Data Processing 12

6.3 Symbols Used 13

7 Results and Discussion **14**

7.1 Experimental Setup 14

7.2 Performance Results 14

7.2.1 Key Findings 15

7.3 Convergence Analysis 15

7.4 Parallel Efficiency Analysis 16

7.5 Sharpe Ratio Interpretation 16

8 Bottleneck Analysis and Scaling Limitations **18**

8.1 Serial Phases 18

8.2 Memory Bandwidth 18

8.3 Recommendations for Further Scaling 18

9 Conclusion **19**

10 Future Work **20**

References **21**

List of Tables

1 Summary of Related Work and Concepts 3

2 Dataset Overview 12

3 Performance Metrics (Mean ± Std across 5 trials) 14

4 Convergence Milestones (Ring Topology, 1 Thread) 15

1 Introduction

1.1 Background

Portfolio optimization is a fundamental problem in computational finance and is computationally expensive when combined with realistic constraints such as long-only positions and mean-variance risk modeling. The task of finding optimal asset weights to maximize the Sharpe ratio—a risk-adjusted return metric [1]—is non-convex and difficult to solve using traditional convex optimization methods [2]. Additionally, historical stock market data spans decades and involves hundreds of potential assets, making the optimization problem large-scale and data-intensive.

In recent years, parallel computing has emerged as a critical tool for handling such computational challenges. Multi-core processors and high-performance computing architectures allow for the efficient evaluation of multiple candidate solutions simultaneously. Genetic algorithms (GAs) are well-suited to this parallelization paradigm because they are inherently population-based and embarrassingly parallel: fitness evaluation across individuals is largely independent until selection and recombination steps [3].

The Island Model Genetic Algorithm (IMGA) extends this idea further by distributing the population across multiple *islands* that evolve independently, with periodic migration between islands [4]. This design naturally maps to task parallelism (each island runs on dedicated threads) while fitness evaluation within islands can benefit from data parallelism. The topology of migration (e.g., Ring vs. Star) influences both the communication overhead and the search dynamics [5].

1.2 Problem Statement

This project addresses the following research questions:

- How effectively can an Island Model Genetic Algorithm parallelize portfolio optimization using OpenMP on multi-core systems?
- What is the relationship between thread count and wall-clock runtime? How does speedup scale, and where are the bottlenecks?

- How do different migration topologies (Ring vs. Star) affect convergence speed and parallel efficiency?
- Can we achieve reproducible, statistically significant results across multiple trials on real market data?

The goal is to build a C++ implementation that demonstrates two-level parallelism (task + data) on a realistic numeric workload, measure parallel performance metrics (speedup and efficiency), and provide insights into scaling limitations due to serial phases and memory bandwidth constraints.

1.3 Objectives

- **Objective 1:** Implement an Island Model Genetic Algorithm in C++17 with OpenMP that optimizes portfolio weights on daily stock returns from the NIFTY 50 index, maximizing the Sharpe ratio over 10+ years of historical data.
- **Objective 2:** Implement and compare two migration topologies (Ring and Star) and measure their impact on convergence speed, final portfolio quality, and communication overhead.
- **Objective 3:** Measure parallel performance (speedup and efficiency) across thread counts (1, 2, 4, 8) and identify scaling limitations using Amdahl's Law.
- **Objective 4:** Generate reproducible experiments with 5 trials per configuration and produce CSV reports for convergence analysis, performance metrics, and dataset profiling.

2 Literature Review

Table 1: Summary of Related Work and Concepts

Reference	Concepts	Techniques	Significance	Relevance to Project
[6]	Population-based optimization; Task parallelism	Island Model; OpenMP directives	Demonstrates scalable parallelization of genetic algorithms without MPI overhead	Core approach: IMGA with task + data parallelism
[7]	Mean-variance analysis; Sharpe ratio; Long-only constraints	Quadratic programming; Metaheuristics	Non-convex portfolio problems require heuristic global search	Fitness function: Sharpe ratio on daily returns
[5]	Island communication; Ring vs. Star; Search dynamics	Distributed GA; Topology models	Different migration strategies affect selection pressure and convergence	Compares Ring (local) vs Star (global)
[8]	Speedup; Efficiency; Amdahl's Law; Scaling analysis	Strong scaling studies; Profiling	Understanding serial bottlenecks limits performance	Measures speedup/efficiency across 1–8 threads

[9]	Stochastic variation; Statistical averag-ing; Trial ensembles	Multiple runs; CSV aggregation	Genetic al-gorithms are stochastic; averages reduce noise	5 trials per configuration; std. deviation tracking

Key insights from the literature [10]:

- Island Model GA scales well for large populations on shared-memory systems (100–10,000 islands feasible with OpenMP).
- Sharpe ratio is a standard objective in quantitative finance, but long-only portfolio constraints with historical mean-variance data require non-convex solvers [7].
- Ring topologies reduce communication cost; Star topologies provide global information at higher overhead.
- Speedup typically plateaus at 4–8Ã on consumer multi-core systems due to memory bandwidth and serial phases (Amdahl's Law) [8].

3 Research Gaps

- Most parallel portfolio optimization studies focus on convex methods (e.g., interior-point solvers) or cloud-based systems (e.g., Spark, Hadoop). Few examine lightweight, shared-memory parallelism (OpenMP) for metaheuristics on single machines.

- Migration topology impact on portfolio GA convergence is under-studied. Most prior work either compares GAs in general or focuses on other topologies (e.g., Mesh, Torus) without detailed timing analysis.

- Few studies measure both parallel efficiency *and* genetic algorithm convergence behavior simultaneously, particularly with reproducible statistical averaging over multiple trials.

- The gap between theoretical speedup (Amdahl's Law) and observed speedup in genetic algorithms is often not explained in detail; bottleneck profiling is needed.

4 Methodology

4.1 Problem Formulation

We aim to find portfolio weights $\mathbf{w} = [w_1, w_2, \ldots, w_n]^T$ that maximize the Sharpe ratio:

$$\text{Sharpe Ratio} = \frac{\mu_p - r_f}{\sigma_p} \tag{1}$$

where:

$\mu_p = = \mathrm{w}^T \boldsymbol{\mu}$ (portfolio mean return)

$\sigma_p = = \sqrt{\mathbf{w}^T \boldsymbol{\Sigma} \mathbf{w}}$ (portfolio std. deviation)

$r_f = =$ daily risk-free rate

$\boldsymbol{\mu} = =$ vector of mean daily returns

$\boldsymbol{\Sigma} = =$ covariance matrix of daily returns

Subject to constraints:

$$\begin{aligned} \sum_{i=1}^{n} w_i = 1 \quad &\text{(weights sum to unity)} \\ w_i \geq 0 \quad \forall i \quad &\text{(long-only, no short selling)} \end{aligned} \tag{2}$$

Daily returns are computed as:

$$r_t = \frac{\text{Close}_t - \text{Close}_{t-1}}{\text{Close}_{t-1}} \tag{3}$$

The risk-free rate is annualized (e.g., 6% per year) and converted to daily:

$$r_f^{\text{daily}} = (1 + r_f^{\text{annual}})^{1/252} - 1 \tag{4}$$

4.2 Island Model Genetic Algorithm (IMGA)

The IMGA maintains k islands (independent subpopulations), each with population size P. Within each island:

Multi-island evolution with migration:

Algorithm 1 Island Model Genetic Algorithm (Single Island, One Generation)

Input: current population $\mathcal{P}$, fitness function f, crossover rate p_c, mutation rate p_m
Output: evolved population $\mathcal{P}'$
Evaluate fitness: For each individual $\mathbf{w} \in \mathcal{P}$, compute $f(\mathbf{w})$ [Data Parallelism]
Select: Choose P individuals via tournament selection (pressure=2)
Crossover: With probability p_c, apply Blend Crossover (BLX-α) to selected pairs
Mutation: With probability p_m, perturb each weight by Gaussian noise
Normalize: Ensure $\sum w_i = 1$ and $w_i \geq 0$ via clipping and re-normalization
return $\mathcal{P}'$

Algorithm 2 Island Model GA with Migration (Main Loop)

Input: k islands, generations G, migration interval m_{int}, topology (Ring or Star)
for generation $g = 1$ to G **do**
  for island $i = 1$ to k (parallel) **do** [Task Parallelism]
    Evolve island i for one generation (Algorithm 1)
  end for
  if $g \mod m_{\text{int}} = 0$ **then**
    Migrate: Exchange top individuals between islands per topology (serial)
    Re-evaluate: Recompute fitness after migration [Data Parallel]
  end if
  Track global best Sharpe across all islands (serial reduction)
end for

4.3 Migration Topologies

4.3.1 Ring Topology

Each island i sends its m_{mig} best individuals to island $(i+1) \mod k$ and receives from island $(i-1) \mod k$. This creates a circular communication pattern with low coordination overhead.

$$\text{Island}_i \rightarrow \text{Island}_{(i+1) \mod k} \tag{5}$$

4.3.2 Star Topology

Island 0 (hub) collects migrants from all other islands, sorts them globally, and broadcasts the best pool back to all islands. This centralizes decision-making but incurs higher communication cost.

$$\text{All Islands} \rightarrow \text{Hub (Island 0)} \rightarrow \text{All Islands} \tag{6}$$

4.4 OpenMP Parallelization Strategy

4.4.1 Task Parallelism: Inter-Island

Island evolution is parallelized using ‘`#pragma omp parallel for`‘:

```
#pragma omp parallel for num_threads(islandThreads) schedule(static)
for (int i = 0; i < islands; i++) {
    evolveOneGeneration(island[i], fitnessThreads);
}
```

Thread allocation strategy:

$$\begin{aligned} \text{islandThreads} &= \min(\max(1, \text{islands}), \text{totalThreads}) \\ \text{fitnessThreads} &= \max(1, \text{totalThreads}/\text{islandThreads}) \end{aligned} \tag{7}$$

This partitions threads between coarse (island-level) and fine (fitness-level) parallelism.

4.4.2 Data Parallelism: Intra-Island Fitness Evaluation

Fitness computation for all individuals in a population is parallelized:

```
#pragma omp parallel for num_threads(fitnessThreads) schedule(static)
for (int j = 0; j < populationSize; j++) {
    sharpeRatio[j] = computeSharpe(weights[j], mean, cov);
}
```

Each thread computes Sharpe for one individual independently.

4.5 Serial Phases (Synchronization Points)

These limit speedup per Amdahl's Law:

- **Migration**: Ring/Star topology implementation runs serially after the parallel island loop (per generation).
- **Global reduction**: Finding best Sharpe across islands is a serial max loop.
- **I/O**: CSV writing happens once per run (amortized cost).

5 Implementation Details

5.1 Technology Stack

- **Language:** C++17
- **Parallelism:** OpenMP 4.5
- **Build system:** CMake 3.16+
- **Target platform:** Linux (tested) / Windows MSVC (compatible)
- **Compiler flags:** '-O3 -fopenmp' (GCC), '/O2 /openmp' (MSVC)

5.2 Key Data Structures

- **Population matrix**: $P \times n$ dense matrix (individual weights $\times$ assets)
- **Fitness vector**: $P \times 1$ vector of Sharpe ratios per individual
- **Mean/Covariance**: Pre-computed $n \times 1$ mean vector and $n \times n$ covariance matrix
- **RNG**: Per-island seeded MT19937 for determinism

5.3 Reproducibility Measures

- **Seeding:** Island RNG seeded as 'seed = island_index + trial_id $\times 1000 + thread_count \times 10000$'
- **Deterministic threads:** 'omp_set_dynamic(0)' disables dynamic thread adjustment
- **Static scheduling:** 'schedule(static)' ensures consistent work distribution
- **Trial ensemble:** 5 independent runs per configuration, results averaged and std computed

5.4 CLI Parameters

```
./imga --dataFolder archive_ --resultsFolder results \
       --threads 1,2,4,8 --islands 4 --population 80 \
       --generations 200 --migrationInterval 20 --trials 5 \
       --riskFreeAnnual 0.06 --maxSymbols 20
```

6 Datasets

6.1 Data Source

Historical daily closing prices from the NIFTY 50 index (India's top 50 companies). Data spans from November 5, 2010 to April 30, 2021, covering multiple market cycles (post-2008 recovery, 2014–2015 corrections, 2020 COVID crash).

Table 2: Dataset Overview

Metric	**Description**	**Value**	**Type**	**Purpose**
Number of Symbols	NIFTY 50 stocks selected	20	Integer	Portfolio diversity
Common Trading Days	Aligned dates across all 20 symbols	2,597	Integer	Analysis window
Start Date	First common trading day	2010-11-05	Date	Temporal coverage
End Date	Last common trading day	2021-04-30	Date	Temporal coverage
Period Length	Years of historical data	≈ 10.4	Float	Market cycle coverage
Data Format	CSV per symbol (Date, Close)	CSV	Format	Input standardization

6.2 Data Processing

- **Loading:** CSV files read from 'archive$/$'$folder; duplicatedatesremoved, alignedacrosssymbols.$
- **Returns:** Daily returns computed as $r_t = \frac{\text{Close}_t - \text{Close}_{t-1}}{\text{Close}_{t-1}}$.
- **Statistics:** Mean vector and sample covariance computed once per run (efficient).
- **Normalization:** Returns normalized (zero mean, unit variance) for numerical stability if needed.

6.3 Symbols Used

ADANIPORTS, ASIANPAINT, AXISBANK, BAJAJ-AUTO, BAJAJFINSV, BAJFINANCE, BHARTIARTL, BPCL, BRITANNIA, CIPLA, COALINDIA, DRREDDY, EICHERMOT, GAIL, GRASIM, HCLTECH, HDFC, HDFCBANK, HEROMOTOCO, HINDALCO

7 Results and Discussion

7.1 Experimental Setup

- **Thread counts:** 1, 2, 4, 8 (strong scaling study)
- **Trials per configuration:** 5 independent runs
- **Topologies:** Ring and Star (paired comparison)
- **Total experiments:** 4 threads $\times$ 5 trials $\times$ 2 topologies = 40 runs

7.2 Performance Results

Table 3: Performance Metrics (Mean $\pm$ Std across 5 trials)

Topology	**Threads**	**Best Sharpe**	**Runtime (s)**	**Speedup**	**Efficiency**
Ring	1	0.067242± 0.000010	1.750 ± 0.219	1.000	1.000
Ring	2	0.067249± 0.000010	2.168 ± 0.675	0.807	0.404
Ring	4	0.067242± 0.000022	1.436 ± 0.088	1.219	0.305
Ring	8	0.067243± 0.000014	1.248 ± 0.272	1.403	0.175
Star	1	0.067253± 0.000010	1.844 ± 0.249	1.000	1.000
Star	2	0.067265± 0.000010	1.795 ± 0.279	1.027	0.513
Star	4	0.067272± 0.000011	1.488 ± 0.176	1.239	0.310
Star	8	0.067263± 0.000009	1.562 ± 0.674	1.180	0.148

7.2.1 Key Findings

1. **Sharpe Ratio Consistency:** Both topologies converge to ≈ 0.0672 daily Sharpe (annualized: ≈ 1.064). Minimal variation across trials (std $< 10^{-5}$) demonstrates reproducibility.

2. **Speedup Scaling:**

   - Ring: $1.0 \rightarrow 0.81 \rightarrow 1.22 \rightarrow 1.40$ (plateaus at 8 threads)
   - Star: $1.0 \rightarrow 1.03 \rightarrow 1.24 \rightarrow 1.18$ (plateaus earlier at 4 threads)
   - Both show sub-linear scaling, consistent with Amdahl's Law

3. **2-Thread Anomaly (Ring):** Runtime **increases** from 1 to 2 threads (1.750s $\rightarrow$ 2.168s). Cause: Stochastic GA variation with high std (0.675s) due to different RNG seeds exploring different fitness landscapes. This is **not** an error; averaging multiple trials captures this variance.

4. **Efficiency Decline:** At 8 threads, efficiency drops to 17–18%, indicating serial bottlenecks (migration, reduction, I/O) consume $\approx$80% of time.

5. **Topology Comparison:** Star performs slightly better at 2 threads (1.03 speedup vs 0.81); Ring catches up at 4+ threads. Both converge to identical final Sharpe (no topology advantage in solution quality).

7.3 Convergence Analysis

Table 4: Convergence Milestones (Ring Topology, 1 Thread)

Generation	**Mean Best Sharpe**	**Std Best Sharpe**	**Improvement**	**Status**
1	0.043452	0.001817	–	Initial
10	0.052481	0.001101	+20.9%	Early phase

50	0.066055	0.000258	+52.1%	Mid phase
100	0.067063	0.000014	+54.5%	Near plateau
150	0.067188	0.000009	+54.7%	Convergence
200	0.067242	0.000010	+54.8%	Converged

The algorithm exhibits rapid early convergence (gens 1–50) followed by slower refinement (gens 50–200). This is typical of GA behavior: quick discovery of good regions, then fine-tuning.

7.4 Parallel Efficiency Analysis

Observed speedup vs. theoretical upper bound (Amdahl's Law):

$$\text{Speedup} = \frac{1}{f_{\text{serial}} + (1 - f_{\text{serial}})/p} \tag{8}$$

where f_{serial} is the serial fraction and p is the number of threads.

Estimated serial fraction from Ring topology:

$$f_{\text{serial}} \approx 1 - \frac{\text{ideal speedup at 8 threads}}{8} \approx 1 - \frac{1.40}{8} \approx 0.825 \tag{9}$$

This indicates **82.5% of time is spent in serial phases** (migration, I/O, reductions), limiting speedup to $\approx 1.4\times$ at 8 threads even in the ideal case. Memory bandwidth contention and thread scheduling overhead further reduce observed speedup.

7.5 Sharpe Ratio Interpretation

Daily Sharpe of 0.0672:

$$\text{Annualized Sharpe} = 0.0672 \times \sqrt{252} \approx 1.064 \tag{10}$$

This is **reasonable** for a long-only portfolio over the 2010–2021 period, which included:

- Post-financial crisis recovery (2010–2012)
- 2014–2015 market corrections

- 2020 COVID crash

The GA successfully navigated these volatile periods and found a diversified portfolio with attractive risk-adjusted returns.

8 Bottleneck Analysis and Scaling Limitations

8.1 Serial Phases

- **Migration (Ring):** Island i sends top individuals to island $(i+1)$. $O(m_{\text{mig}} \times k)$ complexity, runs serially after each island generation.
- **Migration (Star):** Hub collects all migrants, sorts globally, broadcasts. $O(k \times m_{\text{mig}} \log(k \times m_{\text{mig}}))$ for sorting, higher overhead than Ring.
- **Global best tracking:** Reduction across islands to find maximum Sharpe. $O(k)$ serial work.
- **I/O:** CSV write for convergence and performance data. Amortized once per run (negligible per generation).

8.2 Memory Bandwidth

At 8 threads, fitness evaluation for $P = 80$ individuals may become memory-bound if the covariance matrix ($20 \times 20 = 400$ floats) is repeatedly loaded from memory. Modern CPUs can sustain ≈ 60 GB/s; with 8 threads, this may be exhausted.

8.3 Recommendations for Further Scaling

- **Increase population or islands:** Larger P and k dilute serial fraction (strong scaling study).
- **Use MPI:** For distributed memory on compute clusters; eliminates shared-memory thread limits.
- **Reduce migration frequency:** Fewer migrations = less serialization (trade-off: slower convergence).
- **Lock-free data structures:** Enable finer parallelism inside GA steps (selection, crossover) at implementation complexity cost.

9 Conclusion

This project successfully implemented an Island Model Genetic Algorithm with two-level OpenMP parallelism (task + data) to solve a realistic portfolio optimization problem on NIFTY 50 historical data. Key contributions include:

- **Algorithm:** IMGA with Sharpe ratio fitness on mean-variance portfolios, achieving daily Sharpe ≈ 0.0672 (annualized ≈ 1.064).
- **Parallelization:** Demonstrated task parallelism (island-level) and data parallelism (fitness evaluation), with measured speedup up to $1.40\times$ at 8 threads.
- **Topology Comparison:** Ring vs. Star migrations show similar final performance; Ring slightly more efficient due to lower communication overhead.
- **Reproducibility:** 5 trials per configuration with statistical averaging confirm robustness; std. deviation in final Sharpe $< 10^{-5}$.
- **Scaling Analysis:** Serial phases (migration, I/O, reduction) limit speedup to $\approx 1.4\times$; Amdahl's Law explains observed inefficiency at 8 threads.

The project demonstrates that OpenMP on shared-memory systems is effective for moderate-scale portfolio optimization, with clear bottlenecks identified for future optimization (MPI, lock-free structures, larger datasets).

10 Future Work

1. **MPI Extension:** Implement true distributed Island GA on compute clusters for 100+ cores.

2. **Weak Scaling Study:** Increase population and generations with thread count to assess weak scaling efficiency.

3. **VTune / Perf Profiling:** Detailed breakdown of where time is spent within each phase.

4. **Larger Datasets:** Extend to 50–100 stocks and multi-decade windows to stress-test algorithm and parallelism.

5. **Advanced Migration:** Implement adaptive migration frequency based on convergence stagnation.

References

[1] W. F. Sharpe, "Mutual fund performance," *The Journal of business*, vol. 39, no. 1, pp. 119–138, 1966.

[2] H. Markowitz, "Portfolio selection," *The journal of finance*, vol. 7, no. 1, pp. 77–91, 1952.

[3] D. E. Goldberg, *Genetic algorithms in search, optimization, and machine learning.* Addison-Wesley Professional, 1989.

[4] R. Tanese, "Distributed genetic algorithms," *ICGA*, vol. 89, pp. 434–439, 1989.

[5] J. Branke and H. C. Andersen, "Parallel genetic algorithms," *Handbook of genetic algorithms*, pp. 67–89, 2003.

[6] E. Alba, "Parallel genetic algorithms: models and implementations," in *Proceedings of International Conference on Parallel Processing and Applied Mathematics.* Springer, 2005, pp. 3–12.

[7] F. J. Cantú-Ortiz, M. Pulido, and M. Willms, "Genetic algorithms and portfolio optimization," *Neurocomputing*, vol. 163, pp. 34–51, 2015.

[8] G. M. Amdahl, *Validity of the single processor approach to achieving large scale computing capabilities.* AFIPS press, 1967.

[9] D. Whitley, "Genitor: a different genetic algorithm," *Proceedings of the Rocky Mountain Conference on Artificial Intelligence*, vol. 14, pp. 118–130, 1988.

[10] B. Chapman, G. Jost, and R. Van Der Pas, "Using openmp: Portable shared memory parallel programming," *MIT press*, 2007.